Software Engineering

Software Engineering

- What is it?

Computer Software is the product that software professionals design, build and support over the long term.

It encompasses programmes that execute within a computer of any size and architecture.

- Who does it?

Software engineers build and support software.

The Nature of Software

- **Software is a product:** Transforms information-produces, manages, acquires, modifies, displays or transmits information.
- Delivers computing potential of hardware and networks.
- **Software is a vehicle for delivering a product:**
- Controls other programmes (OS)
- Effects communications (N/W S/w)
- Helps build other software (tools, environments)

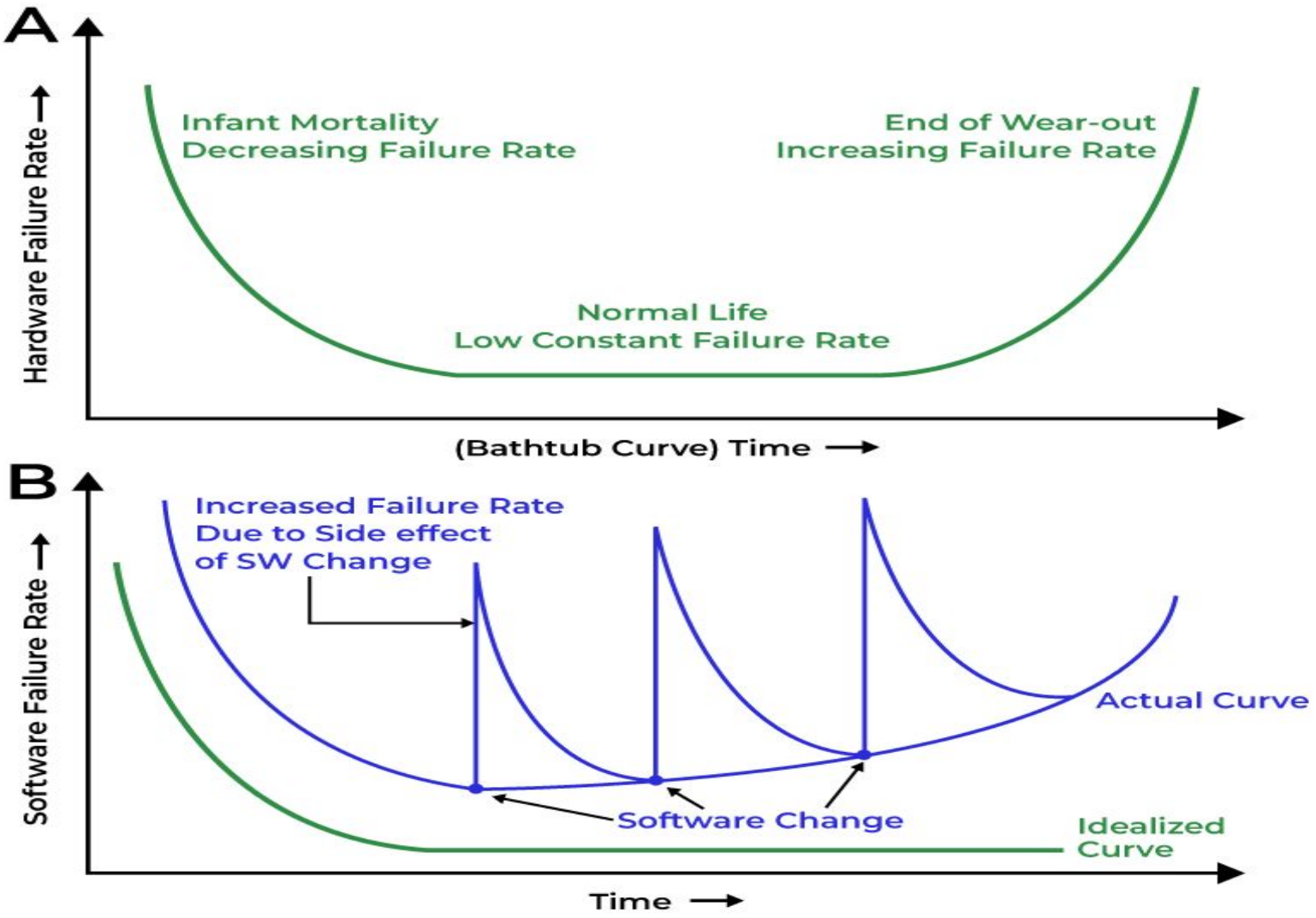
Defining Software

- Software is: (1) ***Instructions*** (Computer Programs) that when executed provide desired features, function, and performance; (2) ***Data structures*** that enable the programs to adequately manipulate information and ***(3) Documentation*** that describes the operation and use of the programs.

S/W vs H/W

- Software is a logical rather than a physical system element.
- Software doesn’t wear out.
- Bathtub Curve: Failure rate as a function of time for Hardware.
- Software is not susceptible to the environmental effects.
- Failure rate curve for software should be “idealized Curve”. Undiscovered defects cause high failure rates early in the life.

Bathtub Curve



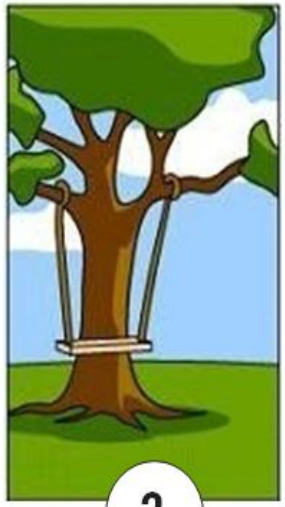
Why to study Software Engineering

Software Development life cycle without Software Engineering



1

How the Customer Explains Requirement



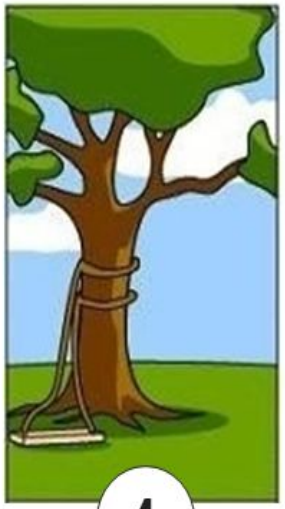
2

How the Project Leader understand it



3

How the System Analyst design it



4

How the Programmer Works on it



5

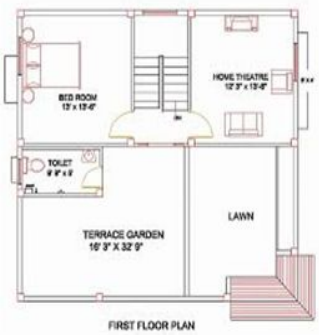
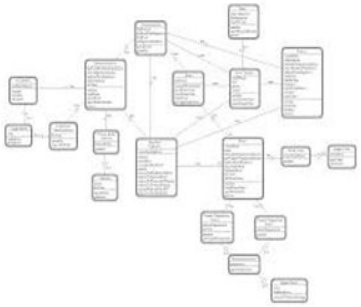
How the Business Consultant describe it

Software Development life cycle without Software Engineering

Customer Requirement	Solution
• Have one trunk • Have four legs • Should carry load both passenger & cargo • Black in color • Should be herbivorous	• Have one trunk • Have four legs • Should carry load both passenger & cargo • Black in color • Should be herbivorous
	 Our value added, also gives milk

The software development process needs to be **engineered** to avoid the **communication gap** and to meet the **actual requirements of customer** within **stipulated budget** **and time**.

Software Engineering

	Design	Build	Product
Engineering			
Software Engineering			

Software is dead….

- The **old School view of Software**
  - You buy it
  - You own it &
  - It's your job to manage it
  - That is coming to an end



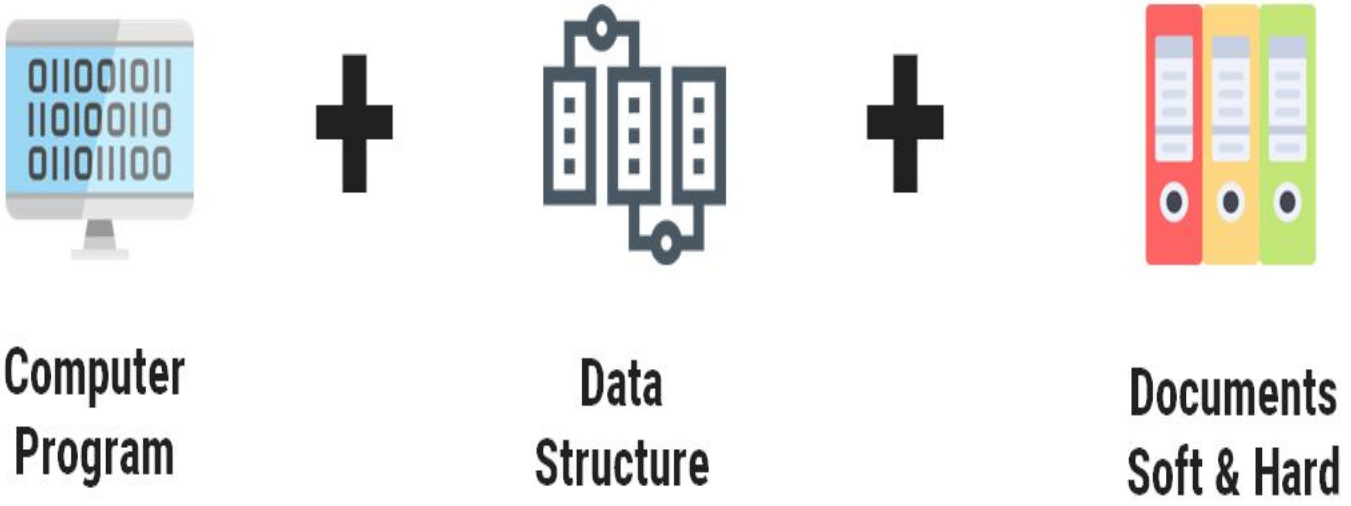
- Because of **web 2.0** & extensive **computing power**, there is a different generation of software
  - It is delivered via Internet
  - It looks exactly like it's residing on each user's computing device
  - Actually it reside on far away server



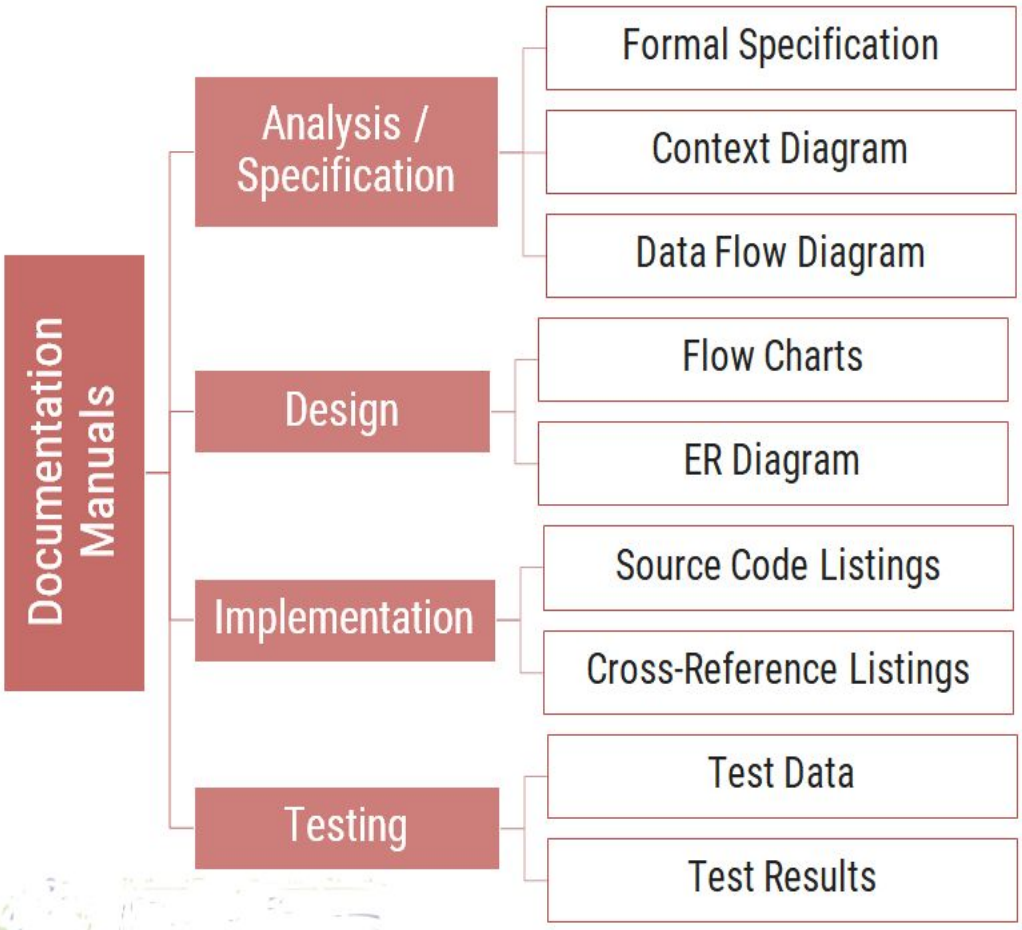


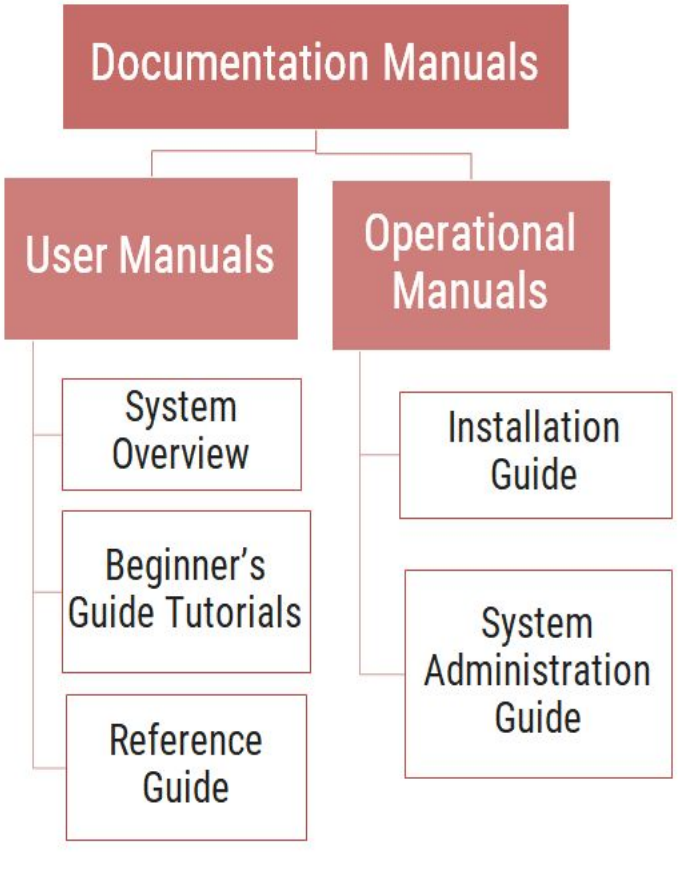
What is Software?

1) **Computer program** that when executed provide desired features, function & performance
2) **Data Structure** that enable programs to easily manipulate information
3) **Descriptive information** in both hard and soft copy that describes the operation and use of programs



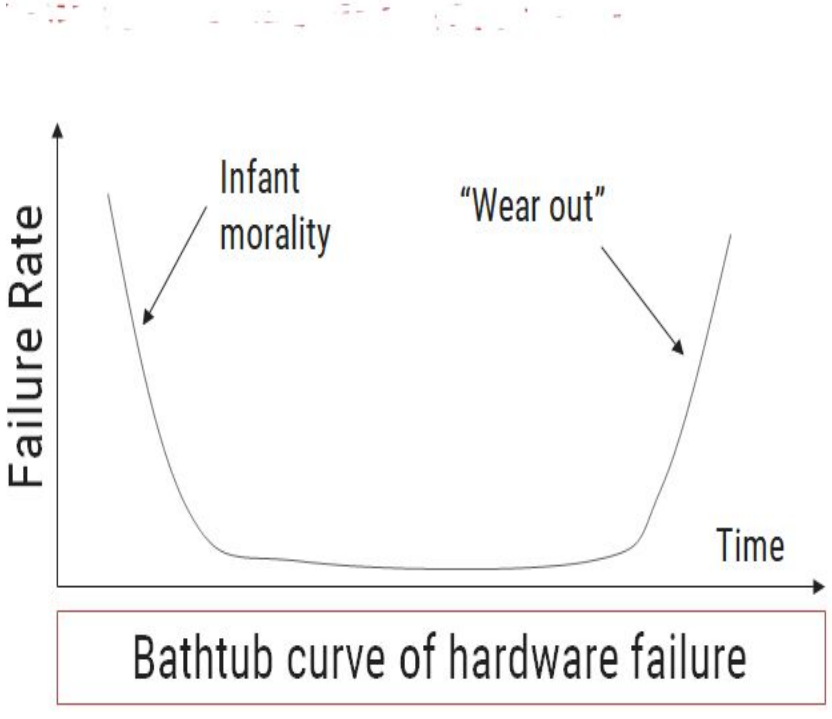
List of Documentation and Manuals



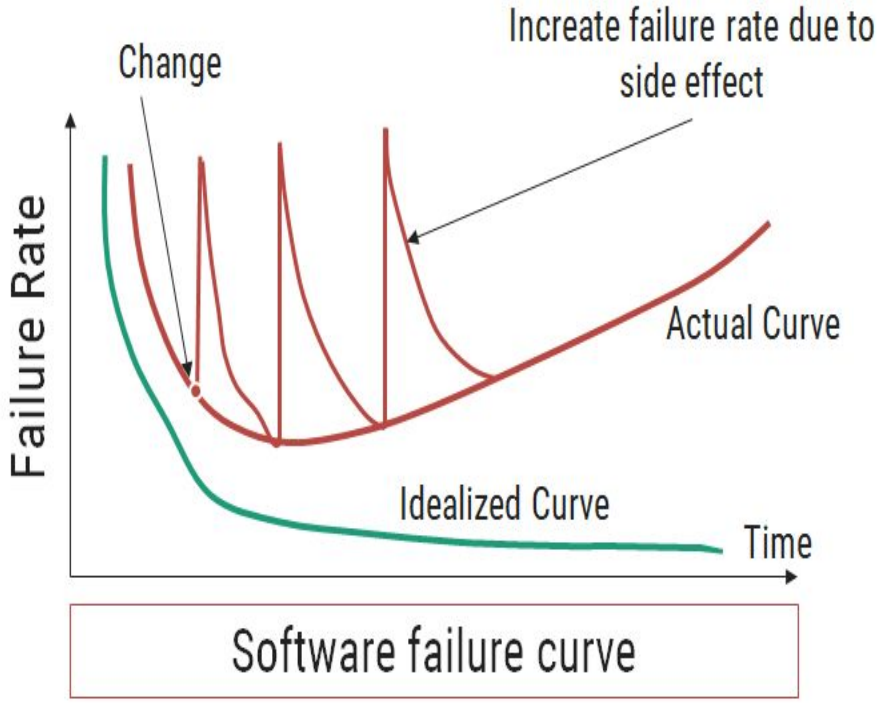


Characteristics of Software

- Software is developed or engineered:
  - It is not manufactured like hardware.
  - Both requires construction of “product” but approaches are different.
- Software doesn’t “wear-out”

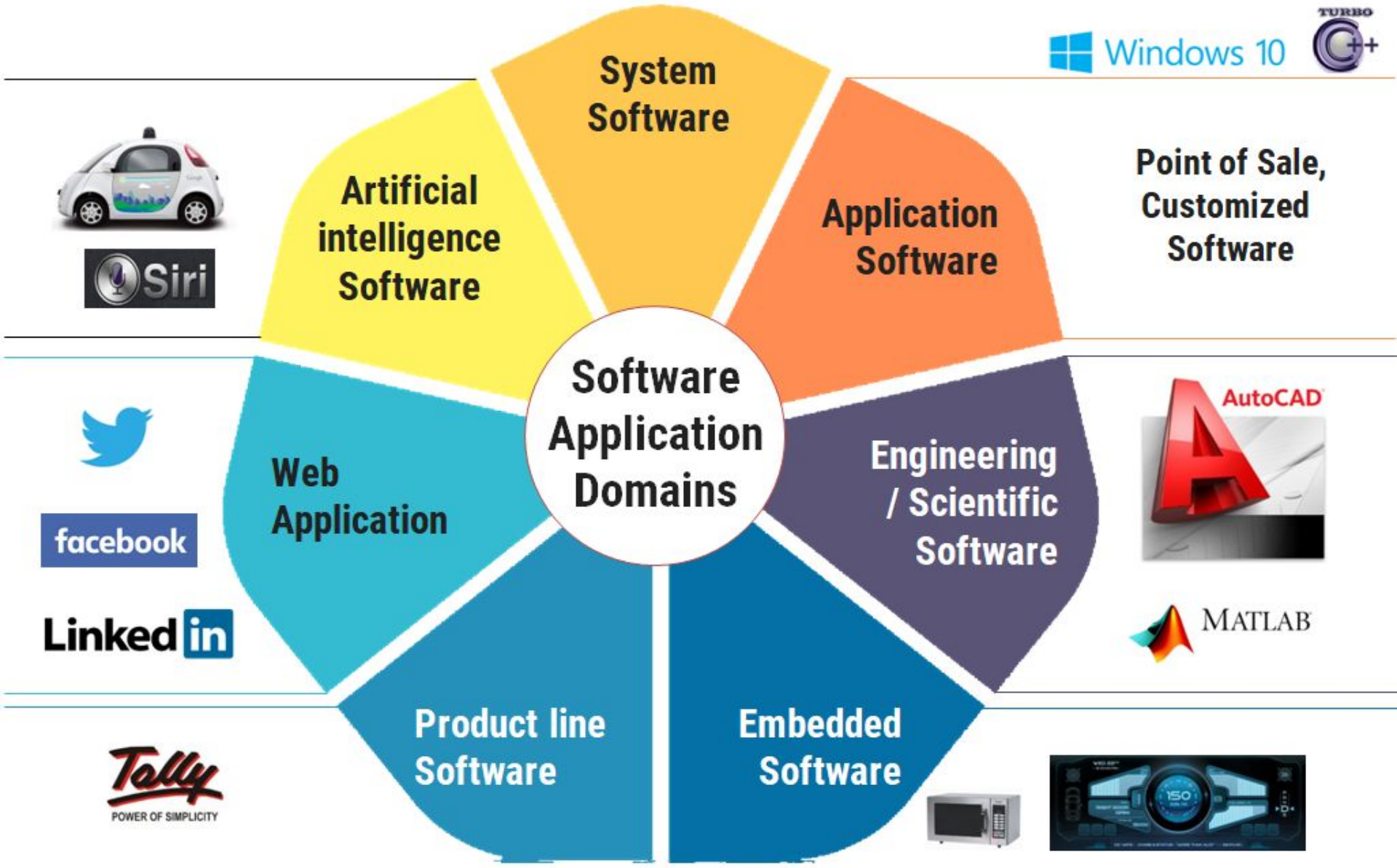


Bathtub curve of hardware failure



Software failure curve

Software Application Domains

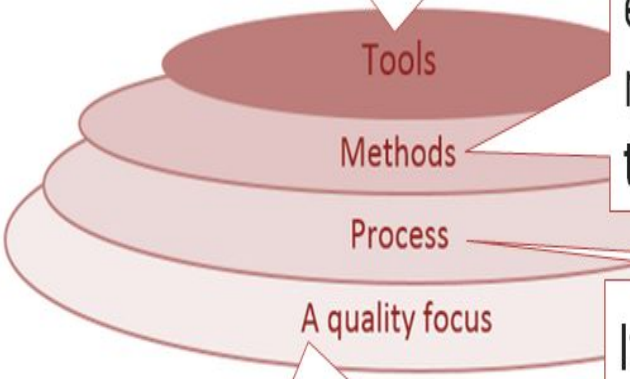


Software Engineering

- Software Engineering is the establishment and use of **sound engineering principles** in order to obtain economically software that is **reliable and works efficiently** in real machines.
- S/W Engg. Is the science and art of building (designing and writing programs) a software systems that are:
  - On **time**
  - On **budget**
  - With acceptable **performance**
  - With **correct operation**.

Software Engineering Layered Approach

Software Engineering Tools **allows automation of activities** which helps to perform systematic activities. A system for the support of software development, called **computer-aided software engineering** (CASE). **Examples:** Testing Tools, Bug/Issue Tracking Tools etc…



It provides **technical how-to's** for building software, it encompasses **many tasks** including communication, requirement analysis, design modeling, program construction, testing and support

It is a foundation of Software Engineering, It is the **glue** the holds the **technology layers**, It **defines** a **framework activities**

Defines continuous **process improvement principles**

Software Engineering Layered Approach..

Quality

- Main principle of Software Engineering is Quality Focus.
- An **engineering approach** must have a **focus on quality**.
- Total Quality Management **(TQM)**, **Six Sigma**, **ISO** 9001, ISO 9000-3, CAPABILITY MATURITY MODEL **(CMM)**, **CMMI** & similar approaches encourages a continuous process improvement culture

Process Layer

- It is a foundation of Software Engineering, It is the glue the holds the technology layers together and enables logical and timely development of computer software.
- It **defines** a **framework** with activities for effective delivery of software engineering technology
- It establish the context in which technical methods are applied, work products (models, documents, data, reports, forms, etc.) are produced, milestones are established, quality is ensured, and change is properly managed.

Software Engineering Layered Approach..

Method

- It provides **technical how-to's** for building software
- It **encompasses many tasks** including communication, requirement analysis, design modeling, program construction, testing and support

Tools

- Software engineering tools provide automated or semiautomated support for the process and the methods
- Computer-aided software engineering (**CASE**) is the scientific application of a **set of tools** and **methods** to a software system which is meant to **result in high-quality, defect-free, and maintainable software products**.
- CASE tools automate many of the activities involved in various life cycle phases.

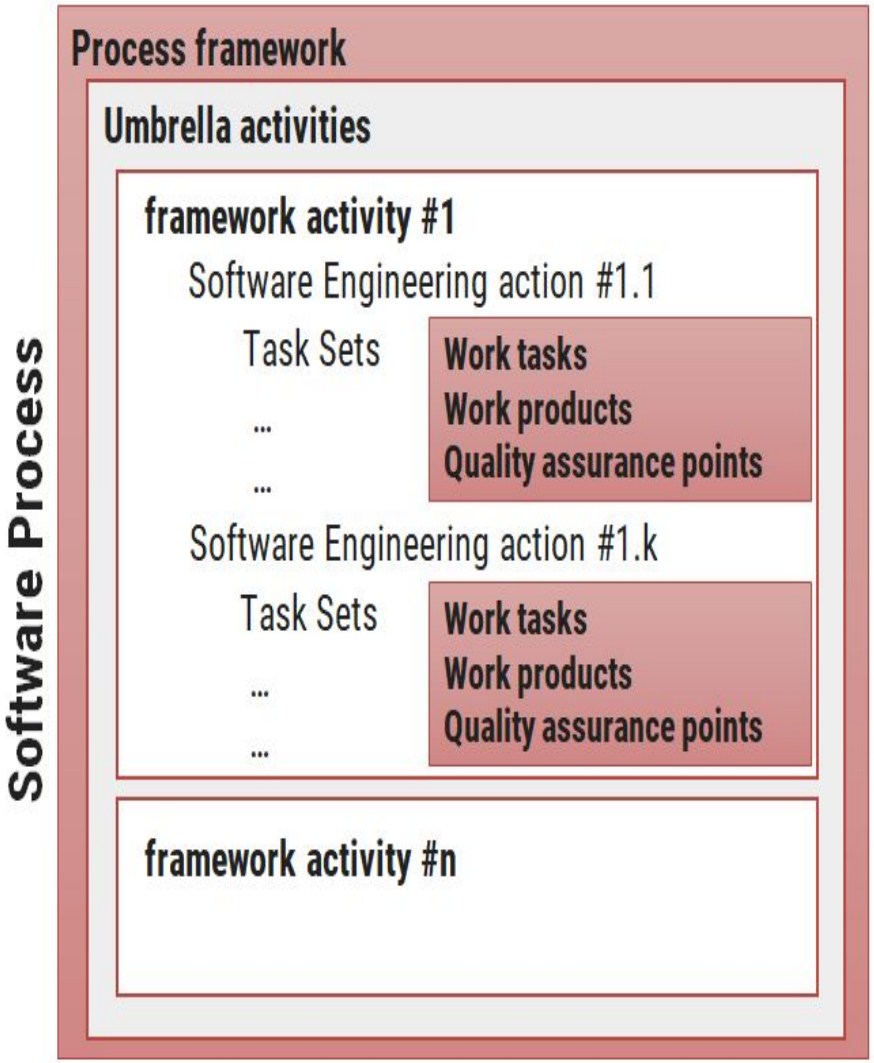
Software Process

- Figure represents "The Software Process"
- Each framework **activity is populated** by **set of** software engineering **actions**
- Each software engineering **action is defined by** a **task set** that identifies work to be completed, product to be produced, quality assurance points & milestones to indicate progress

The **purpose** of software process is

- to **deliver** software in **timely** manner and
- within sufficient **quality to satisfy** those
  - Who has given proposal for software development and
  - Those who will use software

Software Process Framework



Software Framework Activities(CPMCD)

A process framework establishes the foundation for complete software engineering process, it encompasses five activities

Communication		Communication with Customers / stockholders to understand project requirements for defining software features	
Planning		Software Project Plan which defines workflow that is to follow. It describes technical task, risks, resources, product to be produced & work schedule	
Modeling		Creating models to understand requirements and shows design of software to achieve requirements	
Construction		Code Generation (manual or automated) & Testing (to uncover errors in the code)	
Deployment		Deliver Software to Customer Collect feedback from customer based on evaluation Software Support	

Software Myths: Beliefs about software and the process used to build it



"**Misleading Attitudes that cause serious problem**" are myths.



Management Myths



Customer Myths



Practitioner's (Developer) Myths

Management myth - 1 & 2

We **have standards and procedures** to build a system, which is enough.



We have **the newest computers and development tools**.

Reality

- Are software **practitioners aware** of standard's existence?
- Does it **reflect modern software engineering** practice?
- Is it **complete**?
- Is it streamlined to **improve time to delivery** while **still maintaining a focus on quality**?
- In many cases, the answer to all of these questions is "no."

Reality

- It **takes much more than the latest model** computers to do high-quality software development.
- **Computer-aided software engineering (CASE)** tools are more important than hardware.

Management myth - 3 & 4

We **can add more programmers** and can catch up the schedule.



I **outsourced the development** activity, now I **can relax** and **can wait** for the **final** working **product**.

Reality

- Software **development is not a mechanistic process** like manufacturing.
- In the words of Fred Brooks : "**adding people to a late software project makes it later.**"
- **People** who were **working** must **spend time educating** the newcomers.
- People can be added but only **in a planned and well-coordinated** manner.

Reality

- If an **organization** does **not understand how to manage** and **control** software projects internally, it will invariably struggle when it outsources software projects.

Customer myth - 1 & 2

A **general statement of objectives** (requirements) is **sufficient** to start a development.



Requirement Changes can be **easily accommodated** because software is very flexible.

Reality

- Comprehensive (**detailed**) **statements** of requirements is not always possible, an **ambiguous** (unclear) "**statement of objectives**" can lead to disaster.
- Unambiguous (clear) requirements can be gathered only through effective and continuous communication between customer and developer.

Reality

- It is true that software **requirements change**, but the **impact** of change **varies with the time** at which it is introduced.
- When requirements changes are requested early the cost impact is relatively small.

Practitioner's (Developer) myth – 1 & 2

Once we **write** the **program**, our **job is done**.



I **can't** access **quality until** it is **running**.

Reality

- Experts say "**the sooner you begin 'writing code', the longer it will take you to get done.**"
- Industry data indicates that 60 to 80 % effort expended on software will be after it is delivered to the customer for the first time.

Reality

- One of the most effective software **quality assurance mechanisms** can be **applied from** the **beginning** of a project - **the technical review**.
- Software reviews are more effective "quality filter" than testing for finding software defects.

Practitioner's (Developer) myth – 3 & 4

Working **program** is the **only deliverable** work **product**.



Software engineering is about **unnecessary** documentation.

Reality

- **A working program** is only one **part of** a **software configuration**.
- A variety of work products (e.g., **models, documents, plans**) provide a foundation for successful engineering and, more important, guidance for software support.

Reality

- Software engineering is not about creating documents. It is about **creating a quality product**.
- Better quality leads to reduced rework. And reduced rework results in faster delivery times.